

DSC 540 - Data Preparation

Weeks 7 & 8 - Term Project: Milestone 3

Manoj K Kola

Libraries Imported

HTML Parsing

- **lxml**: High-performance XML and HTML parser used by pandas for reading HTML tables
- **html5lib**: Alternative HTML parser that provides better compatibility with modern web pages
- **StringIO**: Utility for reading HTML content as a file-like object in memory

Data Processing

- **pandas**: Primary library for data manipulation, analysis, and working with structured data
- **numpy**: Fundamental package for numerical computing and array operations

Web Requests

- **requests**: Simple HTTP library for sending requests to websites and retrieving web content

Usage

These libraries form the foundation for web scraping workflows, enabling you to:

- Fetch HTML content from websites
- Parse and extract data from HTML tables
- Clean and manipulate the extracted data
- Perform numerical analysis on the results

In [184...

```
# Import lxml parser (used by pandas to read HTML tables)
import lxml

# Import html5lib parser (alternative HTML parser)
import html5lib

# Import pandas for data manipulation and analysis
import pandas as pd

# Import numpy for numerical operations
import numpy as np
```

```
# Import requests to send HTTP requests to websites
import requests

# Import StringIO to read HTML content as a file-like object
from io import StringIO
```

Read HTML tables from the Wikipedia page & Cleaning/Formatting Website Data

This code demonstrates how to extract tabular data from a Wikipedia page using Python's `requests` and `pandas` libraries.

Code Overview

The code below scrapes the Wikipedia page listing the busiest airports in the United States and extracts all HTML tables present on that page.

Step-by-Step Breakdown

1. **Define the target URL:** We specify the Wikipedia page containing airport traffic data
2. **Set request headers:** Browser headers are added to prevent the request from being blocked
3. **Fetch the webpage:** A GET request retrieves the HTML content
4. **Parse HTML content:** The response is converted into a format compatible with pandas
5. **Extract tables:** `pd.read_html()` automatically identifies and extracts all tables from the page
6. **Verify extraction:** The number of tables found is printed for confirmation

Key Components

- **URL:** Wikipedia page for busiest U.S. airports
- **User-Agent header:** Mimics a browser request to avoid potential blocking
- `pd.read_html()` : Pandas function that automatically parses HTML tables into DataFrames
- **Output:** A list of DataFrames, each representing a table from the webpage

Result

The script outputs the total number of tables extracted, which can then be individually examined and analyzed.

In [185...

```
# Define the Wikipedia URL containing the airport data
url = "https://en.wikipedia.org/wiki/List_of_the_busiest_airports_in_the_United_Sta

# Create headers to mimic a browser request (helps avoid request blocking)
headers = {"User-Agent": "Mozilla/5.0"}

# Send a GET request to the website and store the response
response = requests.get(url, headers=headers)
```

```

# Convert the HTML response text into a readable format for pandas
html = StringIO(response.text)

# Read all tables from the HTML page into a list of DataFrames
tables = pd.read_html(html)

# Count the total number of tables extracted from the Wikipedia page
num_tables = len(tables)

# Print the total number of tables found
print("Total tables extracted from the Wikipedia page:", num_tables)

```

Total tables extracted from the Wikipedia page: 10

Display Multiple Tables

This code snippet demonstrates how to loop through and display the first 3 tables from a collection of DataFrames in a clean, readable format.

Key Components:

- Uses `IPython.display` to render tables nicely in Jupyter
- Iterates through the first 3 tables with enumeration for tracking
- Adds clear separators between each table for better readability

```

In [186... # Import display function to nicely show tables in Jupyter Notebook
from IPython.display import display

# Loop through the first 3 tables from the list of tables
for i, table in enumerate(tables[:3]):

    # Print the table number for reference
    print(f"\n===== Table {i} =====")

    # Display the current table in a readable format
    display(table)

```

===== Table 0 =====

	Rank (2024)	Airports (large)	IATA Code	Major cities served	Metro area	State	2024[2]	2023[3]
0	1	Hartsfield–Jackson Atlanta International Airport	ATL	Atlanta	Metro Atlanta	GA	52511402	5095006
1	2	Dallas/Fort Worth International Airport	DFW	Dallas and Fort Worth	Dallas–Fort Worth	TX	42351316	3924621
2	3	Denver International Airport	DEN	Denver	Greater Denver	CO	40012895	3786396
3	4	O'Hare International Airport	ORD	Chicago	Chicago	IL	38575693	3584310
4	5	Los Angeles International Airport	LAX	Los Angeles	Greater Los Angeles	CA	37760834	4095667
5	6	John F. Kennedy International Airport	JFK	New York City	New York Metro	NY	31466102	3080435
6	7	Charlotte Douglas International Airport	CLT	Charlotte	Greater Charlotte	NC	28523822	2589622
7	8	Harry Reid International Airport	LAS	Las Vegas	Las Vegas Valley	NV	28244966	2789619
8	9	Orlando International Airport	MCO	Orlando	Greater Orlando	FL	27859783	2803320
9	10	Miami International Airport	MIA	Miami	Miami Metro	FL	26588002	2471704
10	11	Phoenix Sky Harbor International Airport	PHX	Phoenix	Metro Phoenix	AZ	25595723	2388050
11	12	Seattle–Tacoma International Airport	SEA	Seattle and Tacoma	Seattle Metro	WA	25414592	2459421
12	13	San Francisco International Airport	SFO	San Francisco	San Francisco Bay Area	CA	25078968	2419115
13	14	Newark Liberty International Airport	EWR	Newark and New York City	New York Metro	NJ	24544320	2457532
14	15	George Bush Intercontinental Airport	IAH	Houston	Greater Houston	TX	23349157	2222884
15	16	Logan International Airport	BOS	Boston	Greater Boston	MA	21090721	1996267
16	17	Minneapolis–Saint Paul International Airport	MSP	Minneapolis and Saint Paul	Minneapolis– Saint Paul	MN	18054481	1701912

	Rank (2024)	Airports (large)	IATA Code	Major cities served	Metro area	State	2024[2]	2023[3]
17	18	Fort Lauderdale– Hollywood International Airport	FLL	Fort Lauderdale and Hollywood	Miami Metro	FL	17096131	1706306
18	19	LaGuardia Airport	LGA	New York City	New York Metro	NY	16715567	1617307
19	20	Detroit Metropolitan Airport	DTW	Detroit	Detroit Metro	MI	16110696	1537860
20	21	Philadelphia International Airport	PHL	Philadelphia	Philadelphia Metro	PA	15102261	1365618
21	22	Salt Lake City International Airport	SLC	Salt Lake City	Wasatch Front	UT	13543570	1290536
22	23	Baltimore/Washington International Airport	BWI	Baltimore and Washington, D.C.	Baltimore metropolitan area	MD	13221461	1284972
23	24	Dulles International Airport	IAD	Washington, D.C.	Washington Metro	VA	13003234	1207357
24	25	San Diego International Airport	SAN	San Diego	Greater San Diego	CA	12780013	1219018
25	26	Ronald Reagan Washington National Airport	DCA	Washington, D.C.	Washington Metro	VA	12750892	1236503
26	27	Tampa International Airport	TPA	Tampa	Tampa Bay area	FL	12075591	1167763
27	28	Nashville International Airport	BNA	Nashville	Greater Nashville	TN	12058688	1122724
28	29	Austin–Bergstrom International Airport	AUS	Austin	Greater Austin	TX	10678073	1083344
29	30	Daniel K. Inouye International Airport	HNL	Honolulu	Oahu	HI	10449022	1014976
30	31	Midway International Airport	MDW	Chicago	Chicago	IL	10360093	1065952

===== Table 1 =====

	Rank (2024)	Airports (medium hubs)	IATA Code	City served	Metro Area	State	2024[2]	2023[3]	2022[4]
0	32	Dallas Love Field	DAL	Dallas	Dallas–Fort Worth	TX	8654991	8559052	7500000
1	33	Portland International Airport	PDX	Portland	Portland	OR	8639088	8123054	7500000
2	34	St. Louis Lambert International Airport	STL	St. Louis	St. Louis	MO	7807362	7307561	6500000
3	35	Raleigh–Durham International Airport	RDU	Raleigh	Research Triangle	NC	7584394	7119040	5500000
4	36	William P. Hobby Airport	HOU	Houston	Houston	TX	7116967	6800329	6500000
5	37	Sacramento International Airport	SMF	Sacramento	Sacramento	CA	6679426	6371910	6000000
6	38	Louis Armstrong New Orleans International Airport	MSY	New Orleans	New Orleans	LA	6537092	6309212	5500000
7	39	Luis Muñoz Marín International Airport	SJU	San Juan	San Juan	PR	6490669	5985001	5000000
8	40	Kansas City International Airport	MCI	Kansas City	Kansas City	MO	5915078	5654068	4500000
9	41	Norman Y. Mineta San José International Airport	SJC	San Jose	San Francisco Bay Area	CA	5822019	5958855	5500000
10	42	San Antonio International Airport	SAT	San Antonio	San Antonio	TX	5466684	5336684	4500000
11	43	Southwest Florida International Airport	RSW	Fort Myers	Southwest Florida	FL	5425293	4963269	5000000
12	44	John Wayne Airport	SNA	Santa Ana	Greater Los Angeles	CA	5370273	5706332	5500000
13	45	Oakland San Francisco Bay Airport	OAK	Oakland	San Francisco Bay Area	CA	5292736	5520812	5500000

	Rank (2024)	Airports (medium hubs)	IATA Code	City served	Metro Area	State	2024[2]	2023[3]	2022[4]
14	46	Indianapolis International Airport	IND	Indianapolis	Indianapolis	IN	5181947	4788376	4690000
15	47	Cleveland Hopkins International Airport	CLE	Cleveland	Cleveland	OH	4950345	4803822	4690000
16	48	Pittsburgh International Airport	PIT	Pittsburgh	Pittsburgh	PA	4862376	4493052	3990000
17	49	Cincinnati/Northern Kentucky International Airport	CVG	Cincinnati	Cincinnati	KY	4500414	4287722	3990000
18	50	John Glenn Columbus International Airport	CMH	Columbus	Columbus	OH	4387395	4095189	3990000
19	51	Palm Beach International Airport	PBI	West Palm Beach	Miami Metro	FL	4127211	3815977	3990000
20	52	Jacksonville International Airport	JAX	Jacksonville	Jacksonville	FL	3749791	3619739	3990000
21	53	Ontario International Airport	ONT	Ontario	Greater Los Angeles	CA	3494554	3181161	2990000
22	54	Kahului Airport	OGG	Kahului	Maui	HI	3415615	3745866	4990000
23	55	Hollywood Burbank Airport	BUR	Burbank	Greater Los Angeles	CA	3383237	3132029	3990000
24	56	Bradley International Airport	BDL	Hartford	Hartford	CT	3285194	3122212	2990000
25	57	Charleston International Airport	CHS	Charleston	Charleston	SC	3115194	3032768	2990000
26	58	Milwaukee Mitchell International Airport	MKE	Milwaukee	Milwaukee	WI	3104054	2959840	2990000
27	59	Ted Stevens Anchorage International Airport	ANC	Anchorage	Anchorage	AK	2767856	2681818	2990000

	Rank (2024)	Airports (medium hubs)	IATA Code	City served	Metro Area	State	2024[2]	2023[3]	2022[4]
28	60	Albuquerque International Sunport	ABQ	Albuquerque	Albuquerque	NM	2690218	2605163	2575163
29	61	Eppley Airfield	OMA	Omaha	Omaha	NE	2583212	2464418	2425418
30	62	Buffalo Niagara International Airport	BUF	Buffalo	Buffalo– Niagara Falls	NY	2499295	2291995	2261995
31	63	Boise Airport	BOI	Boise	Boise	ID	2475370	2369164	2339164

===== Table 2 =====

Rank		Airport name	Location	IATA Code	2024[12]	2023[13]	2022[14]	2021[1
0	1	John F. Kennedy International Airport	Queens, New York	JFK	34765387	32481057	25178878	124661
1	2	Los Angeles International Airport	Los Angeles, California	LAX	23438418	21503227	15940358	78625
2	3	Miami International Airport	Miami, Florida	MIA	23202731	21382328	18985551	115924
3	4	San Francisco International Airport	South San Francisco, California	SFO	15083931	13224356	9298781	31390
4	5	Newark Liberty International Airport	Newark, New Jersey	EWR	14690370	14080779	11344732	62508
5	6	O'Hare International Airport	Chicago, Illinois	ORD	13865434	13152578	10746991	51484
6	7	Hartsfield–Jackson Atlanta International Airport	College Park, Georgia	ATL	13813088	12088444	9543318	54742
7	8	Dallas/Fort Worth International Airport	Grapevine, Texas	DFW	11841330	10691190	8992922	58523
8	9	George Bush Intercontinental Airport	Houston, Texas	IAH	11675134	11279214	9124043	64584
9	10	Washington Dulles International Airport	Dulles, Virginia	IAD	10060851	8963807	6690447	32300
10	11	General Edward Lawrence Logan International Ai...	Boston, Massachusetts	BOS	8491518	7651625	5459853	20465
11	12	Orlando International Airport	Orlando, Florida	MCO	7610776	6696949	5214203	18377
12	13	Fort Lauderdale–Hollywood International Airport	Fort Lauderdale, Florida	FLL	6897993	7430312	6125500	40165
13	14	Seattle–Tacoma International Airport	SeaTac, Washington	SEA	6056779	5517422	4004180	13936
14	15	Charlotte Douglas International Airport	Charlotte, North Carolina	CLT	4623143	4140531	3183695	19897
15	16	Denver International Airport	Denver, Colorado	DEN	4444671	3823871	3222132	18561
16	17	Philadelphia International Airport	Philadelphia, Pennsylvania	PHL	3704176	3518548	2613240	9887

	Rank	Airport name	Location	IATA Code	2024[12]	2023[13]	2022[14]	2021[1
17	18	Harry Reid International Airport	Paradise, Nevada	LAS	3592102	3133224	2439479	7382
18	19	Daniel K. Inouye International Airport	Honolulu, Hawaii	HNL	3533266	3134548	1565985	1675
19	20	Detroit Metropolitan Wayne County Airport	Romulus, Michigan	DTW	3353501	2835905	2092098	9663
20	21	Minneapolis–Saint Paul International Airport	Saint Paul, Minnesota	MSP	3320850	2817694	1942385	6737
21	22	Phoenix Sky Harbor International Airport	Phoenix, Arizona	PHX	2821736	2481029	1957189	12238
22	23	LaGuardia Airport	Queens, New York	LGA	1686395	1793314	1334733	3092
23	24	Luis Muñoz Marín International Airport	San Juan, Puerto Rico	SJU	1458172	1158996	802613	4682
24	25	Tampa International Airport	Tampa, Florida	TPA	1384192	1178288	780245	1678
25	26	Antonio B. Won Pat International Airport	Tamuning, Guam	GUM	1342793	1192615	557359	986
26	27	Salt Lake City International Airport	Salt Lake City, Utah	SLC	1311617	1250372	846680	5787
27	28	Baltimore/Washington International Airport	Linthicum Heights, Maryland	BWI	1156882	1201948	1006908	6516
28	29	Austin–Bergstrom International Airport	Austin, Texas	AUS	1064668	987214	818072	2308
29	30	San Diego International Airport	San Diego, California	SAN	995537	904162	538005	2002

Table Selection from Web Scraping Results

This code extracts the first three tables from a list of scraped tables and assigns them to separate DataFrames for further analysis.

Purpose:

- Separates multiple tables scraped from a web page into individual DataFrame objects
- Enables independent manipulation and analysis of each table
- Uses zero-based indexing to access tables from the `tables` list

Variables Created:

- `df_table0` : First table from the scraped data
- `df_table1` : Second table from the scraped data
- `df_table2` : Third table from the scraped data

```
In [187... # Select the first table from the list of scraped tables
df_table0 = tables[0]

# Select the second table from the list
df_table1 = tables[1]

# Select the third table from the list
df_table2 = tables[2]
```

Replace Headers

Data Preparation: Column Renaming

This section cleans up the column names in our airport passenger data to make them more consistent and easier to work with in our analysis.

Steps:

1. **Inspect Original Columns:** First, we examine the current column names to understand the data structure
2. **Rename Columns:** We apply systematic renaming to:
 - Simplify rank and location columns
 - Replace spaces with underscores for programmatic access
 - Remove footnote references from year columns
 - Create a consistent naming pattern for passenger count columns (2015-2024)
3. **Verify Changes:** Finally, we confirm the column names were updated correctly

Key Changes:

- `'Rank (2024)'` → `'Rank'`
- `'Airports (large)'` → `'Airport'`
- `'IATA Code'` → `'IATA_Code'`
- `'Major cities served'` → `'City'`
- `'Metro area'` → `'Metro_Area'`
- `'2024[2]'` → `'Passengers_2024'` (and similar for all years 2015-2024)

This standardized naming convention will make our subsequent data analysis and visualization code cleaner and more readable.

```
In [188... # Display the column names of the dataframe
df_table0.columns
```

```
Out[188... Index(['Rank (2024)', 'Airports (large)', 'IATA Code', 'Major cities served',
      'Metro area', 'State', '2024[2]', '2023[3]', '2022[4]', '2021[5]',
      '2020[6]', '2019[7]', '2018[8]', '2017[9]', '2016[10]', '2015[11]'],
      dtype='object')
```

```
In [189... # Rename the columns to make them cleaner and easier to use
df_table0 = df_table0.rename(columns={
    'Rank (2024)': 'Rank',           # Simplify rank column name
    'Airports (large)': 'Airport',   # Rename airport column
    'IATA Code': 'IATA_Code',        # Replace space with underscore
    'Major cities served': 'City',    # Shorten column name
    'Metro area': 'Metro_Area',       # Replace space with underscore
    'State': 'State',                 # Keep state column name consistent
    '2024[2]': 'Passengers_2024',     # Remove footnote and rename
    '2023[3]': 'Passengers_2023',
    '2022[4]': 'Passengers_2022',
    '2021[5]': 'Passengers_2021',
    '2020[6]': 'Passengers_2020',
    '2019[7]': 'Passengers_2019',
    '2018[8]': 'Passengers_2018',
    '2017[9]': 'Passengers_2017',
    '2016[10]': 'Passengers_2016',
    '2015[11]': 'Passengers_2015'
})

# Display the updated column names to verify the changes
df_table0.columns
```

```
Out[189... Index(['Rank', 'Airport', 'IATA_Code', 'City', 'Metro_Area', 'State',
      'Passengers_2024', 'Passengers_2023', 'Passengers_2022',
      'Passengers_2021', 'Passengers_2020', 'Passengers_2019',
      'Passengers_2018', 'Passengers_2017', 'Passengers_2016',
      'Passengers_2015'],
      dtype='object')
```

Replace Headers

Data Preparation: Column Renaming

This section displays the original column names from `df_table1` and renames them for improved readability and consistency in analysis.

Steps:

1. **Display original columns:** View the current column structure
2. **Rename columns:** Apply standardized naming conventions
 - Remove year references from rank/airport columns
 - Add `Passengers_` prefix to yearly data columns
 - Replace spaces with underscores for programmatic access
3. **Verify changes:** Confirm the updated column names

Naming Conventions Applied:

- **Geographic data:** `Airport`, `IATA_Code`, `City`, `Metro_Area`, `State`

- **Passenger data:** Passengers_YYYY format (e.g., Passengers_2024)
- **Ranking:** Simplified to Rank

This standardization makes the dataset easier to analyze and visualize.

```
In [157... # Display the column names of df_table1
df_table1.columns
```

```
Out[157... Index(['Rank (2024)', 'Airports (medium hubs)', 'IATA Code', 'City served',
      'Metro Area', 'State', '2024[2]', '2023[3]', '2022[4]', '2021[5]',
      '2020[6]', '2019[7]', '2018[8]', '2017[9]', '2016[10]', '2015[11]'],
      dtype='object')
```

```
In [190... # Rename columns to make them cleaner and easier to work with
df_table1 = df_table1.rename(columns={
    'Rank (2024)': 'Rank',
    'Airports (medium hubs)': 'Airport',
    'IATA Code': 'IATA_Code',
    'City served': 'City',
    'Metro Area': 'Metro_Area',
    'State': 'State',
    '2024[2]': 'Passengers_2024',
    '2023[3]': 'Passengers_2023',
    '2022[4]': 'Passengers_2022',
    '2021[5]': 'Passengers_2021',
    '2020[6]': 'Passengers_2020',
    '2019[7]': 'Passengers_2019',
    '2018[8]': 'Passengers_2018',
    '2017[9]': 'Passengers_2017',
    '2016[10]': 'Passengers_2016',
    '2015[11]': 'Passengers_2015'
})

# Display the updated column names to confirm changes
df_table1.columns
```

```
Out[190... Index(['Rank', 'Airport', 'IATA_Code', 'City', 'Metro_Area', 'State',
      'Passengers_2024', 'Passengers_2023', 'Passengers_2022',
      'Passengers_2021', 'Passengers_2020', 'Passengers_2019',
      'Passengers_2018', 'Passengers_2017', 'Passengers_2016',
      'Passengers_2015'],
      dtype='object')
```

Replace Headers

Data Preparation: Column Inspection and Renaming

This section displays the original column names from `df_table2` and renames them for improved readability and consistency in analysis.

Steps:

1. **Display original columns:** View the current column structure
2. **Rename columns:** Apply standardized naming conventions

- Remove year references from rank/airport columns
- Add `Passengers_` prefix to yearly data columns
- Replace spaces with underscores for programmatic access

3. **Verify changes:** Confirm the updated column names

Naming Conventions Applied:

- **Geographic data:** `Airport`, `IATA_Code`, `City`
- **Passenger data:** `Passengers_YYYY` format (e.g., `Passengers_2024`)

This standardization makes the dataset easier to analyze and visualize.

```
In [192...] # Display the column names of the dataframe 'df_table2.'
df_table2.columns
```

```
Out[192...] Index(['Rank', 'Airport', 'City', 'IATA_Code', 'Passengers_2024',
      'Passengers_2023', 'Passengers_2022', 'Passengers_2021',
      'Passengers_2020', 'Passengers_2019'],
      dtype='object')
```

```
In [191...] # Rename columns for better readability and consistency
# - Original column names from the Wikipedia table are replaced with more descriptive
df_table2 = df_table2.rename(columns={
    'Rank': 'Rank',                # Keep the rank column as is
    'Airport name': 'Airport',     # Rename 'Airport name' to 'Airport'
    'Location': 'City',           # Rename 'Location' to 'City'
    'IATA Code': 'IATA_Code',     # Rename 'IATA Code' to 'IATA_Code'
    '2024[12]': 'Passengers_2024', # Rename 2024 column to 'Passengers_2024'
    '2023[13]': 'Passengers_2023', # Rename 2023 column to 'Passengers_2023'
    '2022[14]': 'Passengers_2022', # Rename 2022 column to 'Passengers_2022'
    '2021[15]': 'Passengers_2021', # Rename 2021 column to 'Passengers_2021'
    '2020[16]': 'Passengers_2020', # Rename 2020 column to 'Passengers_2020'
    '2019[17]': 'Passengers_2019'  # Rename 2019 column to 'Passengers_2019'
})

# Display the updated column names to verify changes
df_table2.columns
```

```
Out[191...] Index(['Rank', 'Airport', 'City', 'IATA_Code', 'Passengers_2024',
      'Passengers_2023', 'Passengers_2022', 'Passengers_2021',
      'Passengers_2020', 'Passengers_2019'],
      dtype='object')
```

Format data into a more readable format

Fix casing and strip extra spaces in textual columns

Data Cleaning: Standardize Text Formatting

This code block cleans and standardizes the text formatting across three airport-related datasets (`df_table0`, `df_table1`, `df_table2`).

Processing Steps:

- **Airport names:** Converted to title case (e.g., "john f kennedy" → "John F Kennedy")
- **City names:** Converted to title case (e.g., "new york" → "New York")
- **State codes:** Converted to uppercase (e.g., "ny" → "NY")
- **Whitespace:** Extra leading/trailing spaces are removed from all the 3 fields

The code safely checks for column existence before applying transformations, ensuring it works even if some datasets have different structures.

```
In [162... # Loop through three datasets: df_table0, df_table1, df_table2
for df in [df_table0, df_table1, df_table2]:

    # If the 'Airport' column exists, convert to a string, remove extra spaces, and
    if 'Airport' in df.columns:
        df['Airport'] = df['Airport'].astype(str).str.strip().str.title()

    # If the 'City' column exists, convert to string, remove extra spaces, and titl
    if 'City' in df.columns:
        df['City'] = df['City'].astype(str).str.strip().str.title()

    # If the 'State' column exists, convert to string, remove extra spaces, and con
    if 'State' in df.columns:
        df['State'] = df['State'].astype(str).str.strip().str.upper()

# Print confirmation that the step is completed
print("Casing fixed and extra spaces removed for all three datasets.")
```

Casing fixed and extra spaces removed for all three datasets.

Replace Special characters like '/' & '-' from the Airport field

Airport Column Cleaning

This code cleans and standardizes the 'Airport' column across multiple dataframes by removing special characters and applying consistent formatting.

What it does

- Iterates through three dataframes: `df_table0`, `df_table1`, and `df_table2`
- Verifies the 'Airport' column exists before processing
- Applies a series of string transformations:
 - Converts all values to strings
 - Replaces forward slashes (/) with spaces
 - Replaces en dashes (–) with spaces
 - Removes extra whitespace
 - Capitalizes the first letter of each word

Example

Before: "jfk/new-york "

After: "Jfk New York"

```
In [163... # Loop through multiple dataframes to clean the 'Airport' column
for df in [df_table0, df_table1, df_table2]:

    # Check if the 'Airport' column exists in the current dataframe
    if 'Airport' in df.columns:

        # Clean and format the 'Airport' column
        df['Airport'] = (
            df['Airport']
            .astype(str)                    # Convert all values to string
            .str.replace('/', ' ', regex=False) # Replace slashes with spaces
            .str.replace('-', ' ', regex=False) # Replace dashes with spaces
            .str.strip()                    # Remove Leading/trailing whitespace
            .str.title()                    # Capitalize each word
        )
```

Handle missing or inconsistent values

Data Cleaning: Handling Missing Values

This code loops through multiple dataframes (`df_table0`, `df_table1`, `df_table2`) to standardize the representation of missing values by replacing dash symbols (-) and empty strings with `NaN` values. This ensures consistency across all tables for subsequent analysis and makes it easier to identify and handle missing data using pandas' built-in methods.

```
In [193... # Loop through each dataframe in the list
for df in [df_table0, df_table1, df_table2]:
    # Replace '-' and empty strings with NaN for consistency
    df.replace({'-': np.nan, '': np.nan}, inplace=True)

# Print confirmation that the step is complete
print("Replaced missing or inconsistent values with NaN in all tables.")
```

Replaced missing or inconsistent values with NaN in all tables.

Identify & Drop Duplicates

Remove Duplicate Rows

Loop through each dataframe and remove any duplicate rows in-place. This ensures data quality by eliminating redundant entries across all three tables.

```
In [165... # Loop through each dataframe in the list
for df in [df_table0, df_table1, df_table2]:
    # Drop duplicate rows in-place for each dataframe
    df.drop_duplicates(inplace=True)

# Print confirmation that duplicates have been removed
print("Dropped duplicate rows (if present) in all three tables.")
```

Dropped duplicate rows (if present) in all three tables.

Calculate a New Column - Percentage Change Across Multiple DataFrames

This code loops through three dataframes (`df_table0` , `df_table1` , `df_table2`) and calculates the year-over-year percentage change in passenger counts between 2024 and 2023.

Key Steps:

- Iterates through each dataframe in the list
- Checks for the presence of both `Passengers_2024` and `Passengers_2023` columns
- Calculates percentage change using the formula: $((2024 - 2023) / 2023) \times 100$
- Adds the result as a new column `Pct_Change_24_23` to each dataframe

Output: A confirmation message is printed once the percentage change column has been successfully added to all applicable tables.

```
In [166... # Loop through a list of dataframes to calculate percentage change
for df in [df_table0, df_table1, df_table2]:

    # Check if both 'Passengers_2024' and 'Passengers_2023' columns exist
    if 'Passengers_2024' in df.columns and 'Passengers_2023' in df.columns:

        # Calculate the percentage change between 2024 and 2023 passengers
        df['Pct_Change_24_23'] = (
            (df['Passengers_2024'] - df['Passengers_2023'])
            / df['Passengers_2023']
        ) * 100 # Multiply by 100 to express as a percentage

    # Print confirmation message
    print("Added percentage change column (2024 vs 2023) to all applicable tables.")
```

Added percentage change column (2024 vs 2023) to all applicable tables.

Formatting Percentage Change Column

This code loops through multiple dataframes (`df_table0` , `df_table1` , `df_table2`) to format the `Pct_Change_24_23` column. For each dataframe:

1. **Checks** if the `Pct_Change_24_23` column exists
2. **Rounds** the values to 2 decimal places
3. **Converts** to string format and appends a '%' symbol

The formatting ensures consistent percentage display across all three tables, making year-over-year changes (2024 vs 2023) easier to read in the output.

```
In [167... # Loop through multiple dataframes to format the 'Pct_Change_24_23' column
for df in [df_table0, df_table1, df_table2]:

    # Check if the column exists in the dataframe
    if 'Pct_Change_24_23' in df.columns:

        # Round the values to 2 decimal places, convert to a string, and add a '%'
        df['Pct_Change_24_23'] = df['Pct_Change_24_23'].round(2).astype(str) + '%'
```

```
# Print confirmation that the formatting was applied
print("Added Pct_Change_24_23 with rounding and % sign.")
```

Added Pct_Change_24_23 with rounding and % sign.

Reset Index for Multiple DataFrames

This code resets the index for three cleaned datasets to ensure sequential row numbering starting from 0.

What it does:

- Iterates through three DataFrames: `df_table0`, `df_table1`, and `df_table2`
- Resets each DataFrame's index to a sequential range starting from 0
- The `drop=True` parameter discards the old index instead of adding it as a column
- The `inplace=True` parameter modifies the DataFrames directly without creating copies
- Prints a confirmation message when complete

```
In [169... # Reset the index for multiple dataframes
# We have three cleaned datasets (df_table0, df_table1, df_table2)
# Resetting the index ensures that the rows are numbered sequentially from 0

for df in [df_table0, df_table1, df_table2]:
    df.reset_index(drop=True, inplace=True)

# Print confirmation that all indices have been reset
print("Reset index for all three cleaned datasets.")
```

Reset index for all three cleaned datasets.

Cleaned Datasets Results

The following section displays the cleaned dataset after preprocessing, and data quality checks have been applied.

```
In [170... # Print a header and display the cleaned dataset
print("\n===== Cleaned Dataset =====\n")
```

===== Cleaned Dataset =====

Large Hub Airports - Table0 - Display DataFrame Contents

The following cell displays the contents of `df_table0`, which contains the data extracted from the table.

```
In [171... # Display the contents of the dataframe 'df_table0' in a Jupyter Notebook
display(df_table0)
```

Rank		Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Passenger_Rate
0	1	Hartsfield Jackson Atlanta International Airport	ATL	Atlanta	Metro Atlanta	GA	52511402	100.00%
	2	Dallas Fort Worth International Airport	DFW	Dallas And Fort Worth	Dallas–Fort Worth	TX	42351316	81.53%
2	3	Denver International Airport	DEN	Denver	Greater Denver	CO	40012895	76.20%
3	4	O'Hare International Airport	ORD	Chicago	Chicago	IL	38575693	73.33%
4	5	Los Angeles International Airport	LAX	Los Angeles	Greater Los Angeles	CA	37760834	71.55%
5	6	John F. Kennedy International Airport	JFK	New York City	New York Metro	NY	31466102	59.84%
6	7	Charlotte Douglas International Airport	CLT	Charlotte	Greater Charlotte	NC	28523822	54.33%
7	8	Harry Reid International Airport	LAS	Las Vegas	Las Vegas Valley	NV	28244966	53.81%
8	9	Orlando International Airport	MCO	Orlando	Greater Orlando	FL	27859783	52.86%
9	10	Miami International Airport	MIA	Miami	Miami Metro	FL	26588002	50.43%
10	11	Phoenix Sky Harbor International Airport	PHX	Phoenix	Metro Phoenix	AZ	25595723	48.72%
11	12	Seattle Tacoma International Airport	SEA	Seattle And Tacoma	Seattle Metro	WA	25414592	48.19%
12	13	San Francisco International Airport	SFO	San Francisco	San Francisco Bay Area	CA	25078968	47.55%

Rank		Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Passenger_Rate
13	14	Newark Liberty International Airport	EWR	Newark And New York City	New York Metro	NJ	24544320	100
14	15	George Bush Intercontinental Airport	IAH	Houston	Greater Houston	TX	23349157	100
15	16	Logan International Airport	BOS	Boston	Greater Boston	MA	21090721	100
16	17	Minneapolis Saint Paul International Airport	MSP	Minneapolis And Saint Paul	Minneapolis–Saint Paul	MN	18054481	100
17	18	Fort Lauderdale Hollywood International Airport	FLL	Fort Lauderdale And Hollywood	Miami Metro	FL	17096131	100
18	19	Laguardia Airport	LGA	New York City	New York Metro	NY	16715567	100
19	20	Detroit Metropolitan Airport	DTW	Detroit	Detroit Metro	MI	16110696	100
20	21	Philadelphia International Airport	PHL	Philadelphia	Philadelphia Metro	PA	15102261	100
21	22	Salt Lake City International Airport	SLC	Salt Lake City	Wasatch Front	UT	13543570	100
22	23	Baltimore Washington International Airport	BWI	Baltimore And Washington, D.C.	Baltimore metropolitan area	MD	13221461	100
23	24	Dulles International Airport	IAD	Washington, D.C.	Washington Metro	VA	13003234	100
24	25	San Diego International Airport	SAN	San Diego	Greater San Diego	CA	12780013	100
25	26	Ronald Reagan Washington National Airport	DCA	Washington, D.C.	Washington Metro	VA	12750892	100

	Rank	Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Pa:
26	27	Tampa International Airport	TPA	Tampa	Tampa Bay area	FL	12075591	
27	28	Nashville International Airport	BNA	Nashville	Greater Nashville	TN	12058688	
28	29	Austin Bergstrom International Airport	AUS	Austin	Greater Austin	TX	10678073	
29	30	Daniel K. Inouye International Airport	HNL	Honolulu	Oahu	HI	10449022	
30	31	Midway International Airport	MDW	Chicago	Chicago	IL	10360093	

Medium Hub Airports - Table1 - Display DataFrame

This cell displays the contents of `df_table1` in an interactive, formatted table in the Jupyter Notebook. The `display()` function provides a rich HTML representation of the DataFrame, making it easier to read and explore the data compared to a simple `print()` statement.

```
In [172... # Display the contents of the dataframe 'df_table1' in a readable format in Jupyter
display(df_table1)
```

	Rank	Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Passenger
0	32	Dallas Love Field	DAL	Dallas	Dallas–Fort Worth	TX	8654991	
1	33	Portland International Airport	PDX	Portland	Portland	OR	8639088	
2	34	St. Louis Lambert International Airport	STL	St. Louis	St. Louis	MO	7807362	
3	35	Raleigh Durham International Airport	RDU	Raleigh	Research Triangle	NC	7584394	
4	36	William P. Hobby Airport	HOU	Houston	Houston	TX	7116967	
5	37	Sacramento International Airport	SMF	Sacramento	Sacramento	CA	6679426	
6	38	Louis Armstrong New Orleans International Airport	MSY	New Orleans	New Orleans	LA	6537092	
7	39	Luis Muñoz Marín International Airport	SJU	San Juan	San Juan	PR	6490669	
8	40	Kansas City International Airport	MCI	Kansas City	Kansas City	MO	5915078	
9	41	Norman Y. Mineta San José International Airport	SJC	San Jose	San Francisco Bay Area	CA	5822019	
10	42	San Antonio International Airport	SAT	San Antonio	San Antonio	TX	5466684	
11	43	Southwest Florida International Airport	RSW	Fort Myers	Southwest Florida	FL	5425293	

	Rank	Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Passenger
12	44	John Wayne Airport	SNA	Santa Ana	Greater Los Angeles	CA	5370273	
13	45	Oakland San Francisco Bay Airport	OAK	Oakland	San Francisco Bay Area	CA	5292736	
14	46	Indianapolis International Airport	IND	Indianapolis	Indianapolis	IN	5181947	
15	47	Cleveland Hopkins International Airport	CLE	Cleveland	Cleveland	OH	4950345	
16	48	Pittsburgh International Airport	PIT	Pittsburgh	Pittsburgh	PA	4862376	
17	49	Cincinnati Northern Kentucky International Airport	CVG	Cincinnati	Cincinnati	KY	4500414	
18	50	John Glenn Columbus International Airport	CMH	Columbus	Columbus	OH	4387395	
19	51	Palm Beach International Airport	PBI	West Palm Beach	Miami Metro	FL	4127211	
20	52	Jacksonville International Airport	JAX	Jacksonville	Jacksonville	FL	3749791	
21	53	Ontario International Airport	ONT	Ontario	Greater Los Angeles	CA	3494554	
22	54	Kahului Airport	OGG	Kahului	Maui	HI	3415615	
23	55	Hollywood Burbank Airport	BUR	Burbank	Greater Los Angeles	CA	3383237	
24	56	Bradley International Airport	BDL	Hartford	Hartford	CT	3285194	

	Rank	Airport	IATA_Code	City	Metro_Area	State	Passengers_2024	Passe
25	57	Charleston International Airport	CHS	Charleston	Charleston	SC	3115194	
26	58	Milwaukee Mitchell International Airport	MKE	Milwaukee	Milwaukee	WI	3104054	
27	59	Ted Stevens Anchorage International Airport	ANC	Anchorage	Anchorage	AK	2767856	
28	60	Albuquerque International Sunport	ABQ	Albuquerque	Albuquerque	NM	2690218	
29	61	Eppley Airfield	OMA	Omaha	Omaha	NE	2583212	
30	62	Buffalo Niagara International Airport	BUF	Buffalo	Buffalo–Niagara Falls	NY	2499295	
31	63	Boise Airport	BOI	Boise	Boise	ID	2475370	

Table2 - City column has the City & State together.

Data Cleaning: Separating City and State Information

Problem

Table2's `City` column contains both city and state information combined together (e.g., "New York, NY").

Solution

To preserve the original data while preparing for separation, we rename the column and ensure it's in the correct format:

1. **Rename the column** from `City` to `City_Original` to preserve the original combined data
2. **Validate data type** to ensure `City_Original` is a Series (not a DataFrame)
3. **Preview the data** to verify the transformation

Next Steps

After this transformation, you can split `City_Original` into separate `City` and `State` columns using string manipulation methods.

```
In [173... # Rename the 'City' column to 'City_Original' to preserve the original city data
df_table2.rename(columns={'City': 'City_Original'}, inplace=True)

# Ensure 'City_Original' is a Series and not a DataFrame
if isinstance(df_table2['City_Original'], pd.DataFrame):
    # If it's a DataFrame, take the first column and replace the original column
    df_table2['City_Original'] = df_table2['City_Original'].iloc[:, 0]

# Display the first 5 values in the 'City_Original' column of df_table2
df_table2['City_Original'].head()
```

```
Out[173... 0          Queens, New York
1      Los Angeles, California
2          Miami, Florida
3  South San Francisco, California
4          Newark, New Jersey
Name: City_Original, dtype: object
```

City and State Column Separation

This code processes the `City_Original` column in `df_table2`, which contains combined city and state information, and separates them into distinct columns.

Steps:

1. **Split the combined column:** The `City_Original` column is split at the first comma into city and state components
2. **Clean the data:** Both city and state values are stripped of extra whitespace and converted to title case
3. **Insert new columns:** `City` and `State` columns are added immediately after `City_Original`
4. **Standardize territories:** Special handling for Guam and Puerto Rico to ensure consistent naming
5. **Convert state names to codes:** Full state names are mapped to their 2-letter abbreviations using a comprehensive state mapping dictionary

Result: The dataframe now contains separate, standardized `City` and `State` columns with proper formatting and state abbreviations.

```
In [175... # Convert 'City_Original' column to string and split at the first comma into two ne
split_cols = df_table2['City_Original'].astype(str).str.split(',', n=1, expand=True)

# Ensure the resulting DataFrame has exactly two columns labeled 0 and 1
split_cols = split_cols.reindex(columns=[0, 1])

# Display the first few rows to check the result
# split_cols.head()

# Clean the 'city' column - Convert to string, remove extra spaces, and capitalize
city_clean = split_cols[0].astype(str).str.strip().str.title()
```

```

# Clean the 'state' column - Convert to string, remove extra spaces, and capitalize
state_clean = split_cols[1].astype(str).str.strip().str.title()

# Find the column index of 'City_Original' in the dataframe
col_index = df_table2.columns.get_loc('City_Original')

# Insert a new column 'City' right after 'City_Original' with the cleaned city value
df_table2.insert(col_index + 1, 'City', city_clean)

# Insert a new column 'State' after the new 'City' column with the cleaned state value
df_table2.insert(col_index + 2, 'State', state_clean)

# Standardize 'State' values for Guam - If the 'State' column has any variations of 'Guam'
df_table2.loc[df_table2['State'] == 'Guam', 'State'] = 'Guam'

# Standardize 'State' values for Puerto Rico - If the 'State' column has any variations of 'Puerto Rico'
df_table2.loc[df_table2['State'] == 'Puerto Rico', 'State'] = 'Puerto Rico'

# Create a mapping dictionary for U.S. state names to their 2-letter codes
state_map = {
    'Alabama': 'AL', 'Alaska': 'AK', 'Arizona': 'AZ', 'Arkansas': 'AR',
    'California': 'CA', 'Colorado': 'CO', 'Connecticut': 'CT', 'Delaware': 'DE',
    'Florida': 'FL', 'Georgia': 'GA', 'Hawaii': 'HI', 'Idaho': 'ID',
    'Illinois': 'IL', 'Indiana': 'IN', 'Iowa': 'IA', 'Kansas': 'KS',
    'Kentucky': 'KY', 'Louisiana': 'LA', 'Maine': 'ME', 'Maryland': 'MD',
    'Massachusetts': 'MA', 'Michigan': 'MI', 'Minnesota': 'MN', 'Mississippi': 'MS',
    'Missouri': 'MO', 'Montana': 'MT', 'Nebraska': 'NE', 'Nevada': 'NV',
    'New Hampshire': 'NH', 'New Jersey': 'NJ', 'New Mexico': 'NM', 'New York': 'NY',
    'North Carolina': 'NC', 'North Dakota': 'ND', 'Ohio': 'OH', 'Oklahoma': 'OK',
    'Oregon': 'OR', 'Pennsylvania': 'PA', 'Rhode Island': 'RI',
    'South Carolina': 'SC', 'South Dakota': 'SD', 'Tennessee': 'TN',
    'Texas': 'TX', 'Utah': 'UT', 'Vermont': 'VT', 'Virginia': 'VA',
    'Washington': 'WA', 'West Virginia': 'WV', 'Wisconsin': 'WI', 'Wyoming': 'WY'
}

# Convert full state names in the 'State' column to 2-letter codes
# If a state name is not in the mapping, keep it as is
df_table2['State'] = df_table2['State'].map(state_map).fillna(df_table2['State'])

```

Data Preview: City Column Parsing

The `City` column in Table2 originally contains both city and state information combined. We've separated these into distinct columns for easier analysis.

Below, we preview the original combined column alongside the newly created `City` and `State` columns to verify the data parsing:

In [178...

```

# Preview selected columns in the dataframe
df_table2[['City_Original', 'City', 'State']].head()

```

Out[178...

	City_Original	City	State
0	Queens, New York	Queens	NY
1	Los Angeles, California	Los Angeles	CA
2	Miami, Florida	Miami	FL
3	South San Francisco, California	South San Francisco	CA
4	Newark, New Jersey	Newark	NJ

Busiest International Airports in the USA - Table2

This section displays the contents of the `df_table2` dataframe, which contains data about the busiest international airports in the United States.

The `display()` function renders the dataframe in an interactive, formatted table view that shows:

- Column headers
- Row data
- Proper alignment and styling

This is particularly useful in Jupyter notebooks, as it provides a more visually appealing output than the standard `print()` function.

In [179...

```
# Display the contents of the dataframe 'df_table2' in a readable format  
display(df_table2)
```

Rank		Airport	City_Original	City	State	IATA_Code	Passengers_2024	F
0	1	John F. Kennedy International Airport	Queens, New York	Queens	NY	JFK	34765387	
1	2	Los Angeles International Airport	Los Angeles, California	Los Angeles	CA	LAX	23438418	
2	3	Miami International Airport	Miami, Florida	Miami	FL	MIA	23202731	
3	4	San Francisco International Airport	South San Francisco, California	South San Francisco	CA	SFO	15083931	
4	5	Newark Liberty International Airport	Newark, New Jersey	Newark	NJ	EWR	14690370	
5	6	O'Hare International Airport	Chicago, Illinois	Chicago	IL	ORD	13865434	
6	7	Hartsfield Jackson Atlanta International Airport	College Park, Georgia	College Park	GA	ATL	13813088	
7	8	Dallas Fort Worth International Airport	Grapevine, Texas	Grapevine	TX	DFW	11841330	
8	9	George Bush Intercontinental Airport	Houston, Texas	Houston	TX	IAH	11675134	
9	10	Washington Dulles International Airport	Dulles, Virginia	Dulles	VA	IAD	10060851	
10	11	General Edward Lawrence Logan International Airport	Boston, Massachusetts	Boston	MA	BOS	8491518	
11	12	Orlando International Airport	Orlando, Florida	Orlando	FL	MCO	7610776	

	Rank	Airport	City_Original	City	State	IATA_Code	Passengers_2024	F
	12	Fort Lauderdale Hollywood International Airport	Fort Lauderdale, Florida	Fort Lauderdale	FL	FLL	6897993	
	13	Seattle Tacoma International Airport	Seatac, Washington	Seatac	WA	SEA	6056779	
	14	Charlotte Douglas International Airport	Charlotte, North Carolina	Charlotte	NC	CLT	4623143	
	15	Denver International Airport	Denver, Colorado	Denver	CO	DEN	4444671	
	16	Philadelphia International Airport	Philadelphia, Pennsylvania	Philadelphia	PA	PHL	3704176	
	17	Harry Reid International Airport	Paradise, Nevada	Paradise	NV	LAS	3592102	
	18	Daniel K. Inouye International Airport	Honolulu, Hawaii	Honolulu	HI	HNL	3533266	
	19	Detroit Metropolitan Wayne County Airport	Romulus, Michigan	Romulus	MI	DTW	3353501	
	20	Minneapolis Saint Paul International Airport	Saint Paul, Minnesota	Saint Paul	MN	MSP	3320850	
	21	Phoenix Sky Harbor International Airport	Phoenix, Arizona	Phoenix	AZ	PHX	2821736	
	22	Laguardia Airport	Queens, New York	Queens	NY	LGA	1686395	
	23	Luis Muñoz Marín International Airport	San Juan, Puerto Rico	San Juan	Puerto Rico	SJU	1458172	

	Rank	Airport	City_Original	City	State	IATA_Code	Passengers_2024	F
24	25	Tampa International Airport	Tampa, Florida	Tampa	FL	TPA	1384192	
25	26	Antonio B. Won Pat International Airport	Tamuning, Guam	Tamuning	Guam	GUM	1342793	
26	27	Salt Lake City International Airport	Salt Lake City, Utah	Salt Lake City	UT	SLC	1311617	
27	28	Baltimore Washington International Airport	Linthicum Heights, Maryland	Linthicum Heights	MD	BWI	1156882	
28	29	Austin Bergstrom International Airport	Austin, Texas	Austin	TX	AUS	1064668	
29	30	San Diego International Airport	San Diego, California	San Diego	CA	SAN	995537	

Ethical Implications of Data Wrangling

Data Transformations and Sources: This project involved several key transformations on data scraped from Wikipedia's "List of the Busiest Airports in the United States" page, including renaming columns, splitting location data into city and state fields, converting passenger counts to numeric values, handling missing values, and calculating year-over-year percentage changes. The data was sourced from Wikipedia, which aggregates information from credible sources like FAA statistics and airport authorities. Data was acquired ethically using standard web scraping practices with appropriate headers, accessing publicly available information without violating Wikipedia's terms of service.

Assumptions and Legal Considerations: Several assumptions were made during data cleaning: missing values were converted to NaN, the Wikipedia data accurately reflected official statistics, city/state parsing logic was correct, and percentage calculations were meaningful despite COVID-19 data collection anomalies. While there are no legal restrictions on using publicly available Wikipedia data for academic analysis, ethical practices require proper attribution and transparent documentation of all transformations.

Risks and Mitigation: The primary risks include potential misrepresentation of airport rankings if source data contained errors, amplification of Wikipedia's crowd-sourced data quality issues, and misleading calculated metrics without proper context. To mitigate these

concerns, all transformations are fully documented and reproducible in the notebook, the Wikipedia source is clearly cited, assumptions are explicitly stated, and the analysis acknowledges limitations of using secondary-source data that may not represent official FAA statistics directly.